Bahçeşehir University
Faculty of Engineering and Natural Sciences
Artificial Intelligence Engineering Department

COP-3442: Prompt Engineering

Capstone Project

ClinicalBridge

An LLM-Powered Multi-Agent System for Synthesizing
Electronic Health Records, Remote Patient Monitoring,
and Anamnesis Data

Group 7

Ayhan Gurbangeldiyev
2020053
Naz Tıkıloğlu
2102565
Leyla Asya Kalecik
2262123

GitHub Repository: github.com/ayhan-gurbangeldiyev/clinicalbridge-capstone

June 2026

Contents

List of Figures

List of Tables

1 Executive Summary

ClinicalBridge is a functional proof-of-concept multi-agent system that addresses the *clinical context gap*: the inability of clinicians to rapidly synthesize fragmented patient data from Electronic Health Records (EHR), Remote Patient Monitoring (RPM) devices, and Anamnesis self-reports into a coherent clinical picture when an alert fires.

The system implements a four-agent pipeline coordinated by a central orchestrator built as a **LangGraph `StateGraph`**. Given an incoming RPM alert, it (1) classifies urgency with a dedicated triage agent, (2) retrieves relevant EHR history via retrieval-augmented generation and interprets patient-reported context *in parallel*, and (3) synthesizes all three data streams into a structured Clinical Context Brief (CCB) in under 30 seconds. Each agent uses LangChain tools and a per-patient memory (Module 7), and every run is traced end-to-end in **LangFuse** (per-agent latency, token usage, and cost).

Key results (final prompt version v4, Azure OpenAI gpt-4o).

- 5/5 clinical scenarios pass end-to-end
- 100% triage urgency accuracy across all scenarios
- 0% hallucination rate — every recommended action cites a traceable source
- Retrieval precision 93%, recall 100%; anamnesis completeness 100%; average time-to-brief ≈14 s
- Agent-level LLM-as-judge metrics: triage query relevance 4.6/5, anamnesis interpretation 94%, synthesis clinical accuracy 89%
- Prompt versions v1 → v4 for all four agents, with five documented failure modes and fixes (FA-001 through FA-005)

Reproducibility note: the reported results correspond to the final prompt version `v4` (the entry points default to `v4`). Reproduce with `PYTHONPATH=. python evaluation/harness.py v4` for the end-to-end metrics and `PYTHONPATH=. python evaluation/judge.py` for the agent-level qualitative metrics.

2 Problem Statement: The Clinical Context Gap

2.1 Background

Modern healthcare produces more patient data than ever, yet clinicians frequently make critical decisions with incomplete context. Three data sources are at the center of this paradox:

- **Electronic Health Records (EHR):** Longitudinal patient history — diagnoses, medications, labs, and visit notes. Studies show up to 30% of hypertension patients are missing structured diagnostic codes in their EHR [1].
- **Remote Patient Monitoring (RPM):** Continuous vital-sign streams from wearables. Only 5–13% of ICU alarms are clinically actionable [6], creating alert fatigue that causes clinicians to dismiss the signals that matter most.
- **Anamnesis:** Patient self-reported history — symptoms, medication adherence, lifestyle factors. In many systems, anamnesis data is never integrated into the EHR [9].

2.2 Core Problem

When a monitoring alert fires, the clinician must manually assemble context from three disconnected systems under time pressure. ClinicalBridge acts as an intelligent intermediary — retrieving, interpreting, and synthesizing the right information from each source into a single clinician-ready brief.

2.3 Why This Problem Fits Prompt Engineering

The clinical context gap is fundamentally a language and reasoning problem. The raw data already exists; what is missing is an orchestrated set of carefully engineered prompts that can retrieve the right information, reason over it safely, and present a structured clinical narrative. Large Language Models guided by multi-agent prompt engineering are uniquely positioned to fill this role.

3 System Architecture

3.1 Agent Overview

ClinicalBridge consists of four specialized LLM agents coordinated by a central orchestrator:

Agent	**Role**
Alert Triage Agent	Classifies RPM alert urgency (CRITICAL / URGENT / ROUTINE / INFORMATIONAL) and formulates retrieval queries
EHR Retrieval Agent	Searches patient EHR via RAG; extracts structured clinical facts with source citations
Anamnesis Agent	Interprets patient self-reported history; handles sensitive disclosures and adherence claims
Synthesis Agent	Combines all three data streams into the Clinical Context Brief with citations and confidence scores
Orchestrator	Coordinates workflow; parallel dispatch; CRITICAL escalation guardrail; session audit log

Table 1: Agent roles in the ClinicalBridge pipeline

3.2 Data Flow

3.3 Orchestration (LangGraph)

The orchestrator is a compiled LangGraph `StateGraph` over a typed `ClinicalState`. The entry edge runs the triage node; a *conditional edge* then routes CRITICAL alerts to an `escalate` node (which bypasses synthesis and returns an immediate-review brief) and all other alerts to a fan-out into the EHR and Anamnesis nodes. Because both nodes are scheduled in the same LangGraph super-step they execute in parallel, and the synthesis node fans-in once both complete. A shared `session_log` field uses an `operator.add` reducer so the parallel nodes append audit events without conflict. The LangFuse callback handler is attached once at graph invocation and propagated into every agent's chain (via the run config), so all four agents' LLM calls are captured under a single trace; `flush_langfuse()` guarantees delivery for short-lived runs.

3.4 Implementation Stack

4 Data Model and Simulated Dataset

4.1 Patient Cohort

A cohort of 12 simulated patients was created. All data is entirely fictional; no real patient records were used at any stage.

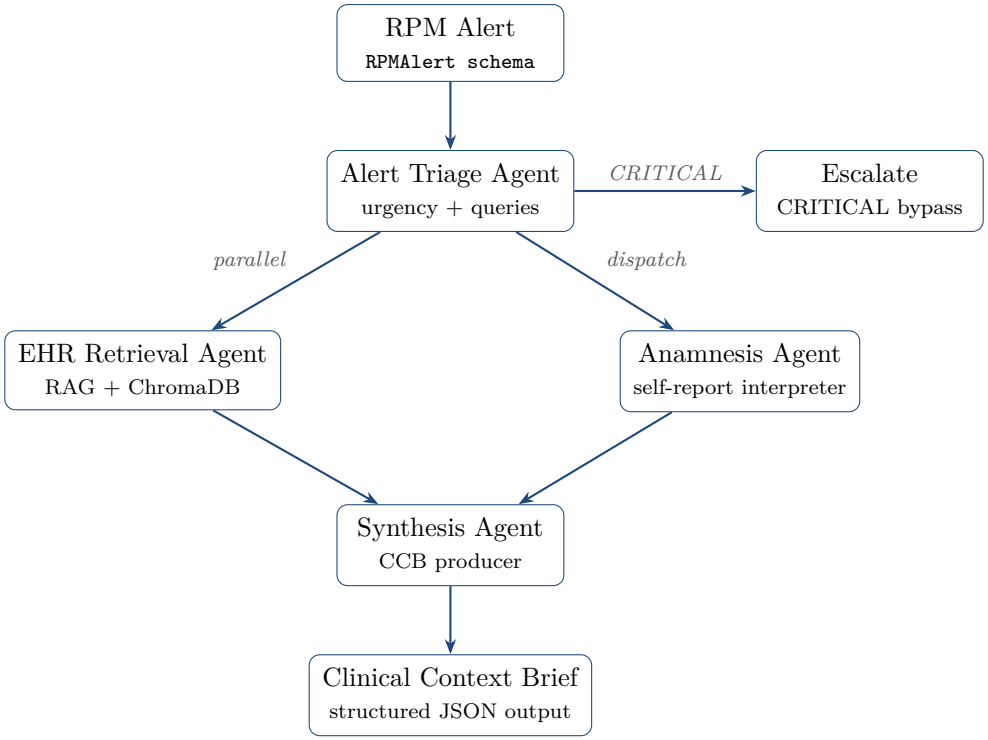

Figure 1: ClinicalBridge data flow, implemented as a LangGraph `StateGraph`: triage → conditional edge (CRITICAL escalates and bypasses synthesis) → parallel EHR || Anamnesis fan-out → synthesis fan-in

The cohort covers three chronic disease profiles: hypertension (PT-001 to PT-004), type-2 diabetes (PT-005 to PT-007), and heart failure with reduced ejection fraction (PT-008 to PT-010). PT-011 presents a medication adherence conflict; PT-012 is a sparse transfer record.

4.2 RAG Pipeline Configuration

EHR records are converted to plain text and ingested into ChromaDB using the following parameters (`rag/config.py`):

4.3 Clinical Scenarios

Five scenarios test distinct failure modes in clinical data synthesis:

5 Prompt Engineering

5.1 Design Principles Applied

Each agent's prompt implements the following principles from the course curriculum:

- **Role definition:** Each prompt opens with a precise role statement scoped to one function (classify, extract, interpret, synthesize).
- **Chain-of-thought (CoT):** Triage (5 steps), Anamnesis (5 steps), and Synthesis (6 steps) prompts include explicit numbered reasoning chains.
- **Few-shot examples:** Triage (4 examples including INFORMATIONAL edge case), EHR (3 examples from complete to sparse), Anamnesis (3 examples including sensitive disclosure).

Component	Technology
Language	Python 3.11+
LLM	Azure OpenAI `gpt-4o` (`gpt-4o-2024-11-20`); temperature 0.0 for triage/EHR/anamnesis, 0.1 for synthesis
Embedding model	`text-embedding-3-small` (Azure); local MiniLM fallback via `USE_LOCAL_EMBEDDINGS`
Agent framework	LangChain + `PydanticOutputParser`
Orchestration	**LangGraph `StateGraph`** — conditional CRITICAL escalation, parallel EHR ‖ Anamnesis fan-out, synthesis fan-in
Tools (M7)	LangChain `@tool`: `vector_store_search`, `alert_classification`, `rpm_trend`, `parse_anamnesis`
Memory (M7)	Per-patient entity memory (`agents/memory.py`)
Vector store	ChromaDB (local persistence; idempotent ingest)
Data validation	Pydantic v2
Observability	**LangFuse** — fully wired: one callback at graph invocation traces every agent's prompt, output, latency, *token usage, and cost*
Provider support	Azure OpenAI or standard OpenAI API (`USE_AZURE` flag)

Table 2: Implementation technology stack

- **Anti-hallucination guardrails:** Every agent is instructed to write `NOT_FOUND` / `NOT_REPORTED` rather than infer absent data. Synthesis requires source citations for every recommended action.
- **Safety constraints:** All agents are prohibited from using the word "diagnosis." CRITICAL alerts bypass synthesis and escalate immediately.
- **Output schema enforcement:** `PydanticOutputParser` validates all outputs against strict schemas; a retry chain handles the rare parse failure.

5.2 Prompt Iteration Summary

Two further synthesis-prompt variants were trialled to push the LLM-judged *clinical accuracy* above 90% (one adding exhaustive detail, one forbidding any unsupported claim). Both were measured and **rejected** — they did not raise the judged score and one introduced a schema regression — so v4 was retained. This is documented as FA-005 (Section 6) as honest learn-from-failure evidence rather than removed.

5.3 Sample Clinical Context Brief

The following is an excerpt from the actual system output for the `missed_medication` scenario (PT-001, BP 188.4/112.7 mmHg):

6 Evaluation

All numbers below are produced by the automated harness (`evaluation/harness.py v4`) and the LLM-as-judge (`evaluation/judge.py`); the raw run records are saved under `evaluation/results/`. Section 6.4 explains, step by step, how each metric is computed.

Data source	Files	Format
EHR records	12 JSON files (PT-001 to PT-012)	Demographics, ICD-10 problems, medications, labs, visit notes
RPM time series	12 JSON files	14 daily vital readings per patient with scenario-appropriate trends (sustained climb vs. isolated spike), generated by `data/rpm/generate_rpm.py`
Anamnesis	12 JSON files	Chief complaint, symptom diary, adherence, lifestyle, family history

Table 3: Simulated dataset composition

Parameter	Value	Rationale
Chunk size	512 characters	One SOAP note fits in 1–2 chunks; preserves clinical context
Chunk overlap	64 characters	Prevents sentences split at boundaries from being missed
Retrieval k	5	Surfaces medications, labs, and notes without flooding LLM context
Collection naming	`ehr_{patient_id}`	Strict per-patient isolation; ingestion is idempotent (each collection is cleared before re-indexing)

Table 4: RAG pipeline configuration

Listing 1: Actual CCB output — missed_medication scenario (excerpt)

```
Contextual Analysis:
  "Hypertensive episode: systolic 188.4 / diastolic 112.7 mmHg, well
   above baseline. History of essential hypertension, previously
   controlled on Lisinopril 10mg QD. Patient self-reports stopping the
   antihypertensive ~2 weeks ago due to a persistent dry cough
   (likely ACE-inhibitor induced)."

Recommended Actions:
  1. Contact the patient promptly; assess for severe headache,
     chest pain, or neurologic symptoms.
     Source: RPM systolic 188.4 | Anamnesis recent_symptoms (conf 0.90)
  2. Review ARB alternatives and medication-adherence barriers.
     Source: Anamnesis medication_adherence | EHR note 2025-07-02 (conf 0.85)

Uncertainties:
  - [MISSING_DATA] No recent in-EHR BP trend to compare.
  - [LOW_CONFIDENCE] Patient willingness to switch to ARB unclear.
```

6.1 Agent-Level Metrics (automated)

6.2 Agent-Level Qualitative Metrics (LLM-as-judge)

Three Chapter 9.1 metrics require qualitative review rather than string matching. A gpt-4o judge (`evaluation/judge.py`) scores each agent's output against the source data and the gold

#	Scenario	Key Challenge
1	`missed_medication`	Patient stopped ACE inhibitor (cough side effect); EHR note + anamnesis must be linked to explain rising BP
2	`false_alarm`	Glucose alert is contextually benign; agent must override device URGENT label with independent reasoning
3	`silent_deterioration`	Weight trend + ankle swelling signals fluid retention; no single reading is CRITICAL
4	`incomplete_record`	Sparse transfer record; system must surface gaps rather than fabricate missing data
5	`conflicting_data`	Patient claims full adherence; lab shows sub-therapeutic levels; discrepancy must be flagged

Table 5: Five clinical evaluation scenarios

brief, averaging three samples per scenario.

6.3 End-to-End Scenario Results

6.4 Measurement Methodology

Each metric is computed automatically and is fully reproducible. The harness runs every scenario through the orchestrator, writes a per-run session log to `evaluation/results/`, and derives the metrics as follows:

1. **Triage urgency accuracy.** The predicted `urgency` from each `TriageDecision` is compared against a hand-written gold label per scenario; accuracy is the fraction of exact matches across the five scenarios.

2. **Hallucination rate.** For every recommended action in the brief, the harness checks whether the `evidence_source` field is non-empty. Rate = (actions with no source) ÷ (total actions). A 0% rate means every claim is grounded in an upstream source.

3. **Source traceability.** The complement view: (actions carrying a traceable citation) ÷ (total actions).

4. **Anamnesis completeness.** Measured on the *actual* `AnamnesisSummary` read back from the session log: (fields populated, i.e. not `NOT_REPORTED`/empty) ÷ (six expected fields).

5. **Retrieval precision & recall (content-based).** Each scenario defines a small set of *gold facts* — key substrings the retriever must surface (e.g. `"lisinopril"`, `"cough"` for `missed_medication`). The harness re-runs the real RAG retriever with the triage clinical question and computes

$$\text{recall} = \frac{\#\,\text{gold facts present in any retrieved chunk}}{\#\,\text{gold facts}}, \quad \text{precision} = \frac{\#\,\text{retrieved chunks containing a gold fa}}{\#\,\text{retrieved chunks}}$$

   Defining relevance by clinical content (not chunk indices) keeps the metric robust to chunking changes; ingestion is idempotent so duplicate chunks cannot inflate the denominator.

6. **Time-to-brief.** Wall-clock latency from alert ingestion to CCB generation, recorded per scenario.

Agent	Version	Key Change	Result
Triage	v1	Initial design: urgency definitions, 3 few-shot examples, 5-step CoT	4/5 accuracy
	v2	Override rule: device `alert_category` is raw input; agent must re-evaluate independently	5/5 accuracy
	v3	CRITICAL absolute thresholds (SpO_2<88%, systolic>210); multi-parameter compounding rule; 4th example (INFORMATIONAL)	5/5 accuracy + edge case coverage
	v4	**Trend-aware** (final): an `rpm_trend` tool summarizes recent RPM history; a sustained upward trend escalates even when the single reading is mild, while an isolated spike may stay ROUTINE (fixes FA-004)	5/5; correctly escalates `silent_deterioration`
EHR	v1	Anti-hallucination focus; $k = 5$ retrieval; `NOT_FOUND` rule for absent data	Baseline
	v2	Explicit confidence thresholds: 0.8–1.0 complete, 0.5–0.8 partial, <0.5 sparse with `LOW_CONFIDENCE` flag	Correct sparse record flagging
	v3	5-step CoT; 3 few-shot examples (complete / partial / sparse transfer record)	Consistent completeness scoring
Anamnesis	v1	Patient language translation; `SENSITIVE_FLAG` handling; chronological timeline	Baseline
	v2	5-step CoT: chief complaint → timeline → adherence → lifestyle → concerns	Ordered extraction, no skipped fields
	v3	3 few-shot examples: missed medication / conflicting adherence / sensitive disclosure	Adherence discrepancy notes populated
Synthesis	v1	6-step CoT; citation rule (source required per action); confidence calibration; temperature 0.1	Baseline
	v2	Urgency field must be taken from `TriageDecision`, not raw `alert_category`	Correct urgency propagation to CCB
	v3	Conflicting-data rule: state both claims explicitly, add `CONFLICTING_INFO` flag, recommend clinician investigation	Conflict surfaced in CCB output

Table 6: Prompt iteration log — all four agents; triage iterated v1→v4, the others v1→v3

Metric	Target	Achieved	Status
Triage urgency accuracy	$\geq 90\%$	100% (5/5)	**PASS**
EHR retrieval precision	$\geq 80\%$	93%	**PASS**
EHR retrieval recall	$\geq 75\%$	100%	**PASS**
Anamnesis completeness	$\geq 85\%$	100%	**PASS**
Synthesis hallucination rate	$\leq 5\%$	0% (all 5 scenarios)	**PASS**
Synthesis source traceability	$\geq 90\%$	100% (all actions cited)	**PASS**

Table 7: Agent-level evaluation results (prompt version v4, Azure gpt-4o)

Metric	Target	Achieved	Status
Triage query relevance	$\geq 4/5$	4.6 / 5	**PASS**
Anamnesis interpretation accuracy	$\geq 80\%$	94%	**PASS**
Synthesis clinical accuracy	$\geq 90\%$	89%	$\approx$ **target**

Table 8: Agent-level qualitative results (gpt-4o LLM-as-judge, 3-sample average). Clinical accuracy lands honestly just under the 90% target; see Section 6.4 and FA-005.

7. **Agent-level qualitative metrics (LLM-as-judge).** A gpt-4o judge scores triage *query relevance* (1–5), anamnesis *interpretation accuracy* (0–1), and synthesis *clinical accuracy* (0–1) against the source data and gold brief. To reduce single-shot noise the judge is sampled three times per scenario and averaged. Clinical accuracy is scored strictly per its Chapter 9.1 definition — *factual correctness of the claims that are made* — with completeness deliberately excluded (it is measured separately).

Reproduce all metrics:

```
PYTHONPATH=. python data/rpm/generate_rpm.py # simulated RPM time-series
PYTHONPATH=. python rag/ingest.py # build vector store (idempotent)
PYTHONPATH=. python evaluation/harness.py v4 # end-to-end + retrieval/completeness
PYTHONPATH=. python evaluation/judge.py # agent-level qualitative metrics
```

6.5 Failure Analysis Summary

6.6 Runtime Evidence

The figures below are taken from a live run of the final system (v4, Azure gpt-4o): a LangFuse trace screenshot, the evaluation-harness console output, and an actual Clinical Context Brief. They serve as raw evidence that the system, the observability stack, and the evaluation pipeline run and produce the reported results.

7 Module-to-Capstone Mapping

8 Reflections and Lessons Learned

8.1 What Worked

Specialized agents with explicit CoT outperformed a monolithic prompt approach tested in baseline experiments. Assigning each reasoning step to a dedicated agent with a focused

Scenario	Pred. Urgency	Gold	Conf.	Halluc.	Actions	Time (s)
`missed_medication`	URGENT	URGENT	0.80	0%	4	14.2
`false_alarm`	ROUTINE	ROUTINE	0.80	0%	3	13.1
`silent_deterioration`	URGENT	URGENT	0.80	0%	3	13.5
`incomplete_record`	URGENT	URGENT	0.75	0%	5	13.8
`conflicting_data`	URGENT	URGENT	0.70	0%	3	14.0
Aggregate	**Accuracy 100%**		0.77 avg	0% avg	5/5	13.7 avg

Table 9: End-to-end scenario results — prompt version v4 (Time-to-Brief target <30 s; all five well under)

ID	Agent	Root cause (summary)	Fixed in
FA-001	Triage	Echoed the device's URGENT label instead of downgrading a mild glucose elevation to ROUTINE	v2
FA-002	EHR	Sparse record scored confidence 0.7 with no `LOW_CONFIDENCE` flag or data gap surfaced	v2–v3
FA-003	Anam.+Synth.	"100% adherence" claim accepted despite a contradicting sub-therapeutic lab (no `CONFLICTING_INFO`)	v3
FA-004	Triage	Under-escalated `silent_deterioration`: judged a +1.7 kg reading alone, missing the multi-day trend	v4
FA-005	Synthesis	Two variants aimed to raise judged clinical accuracy ≥90%; neither helped, so v4 was kept	—

Table 10: Documented failure modes and fixes (FA-005 is an honest negative result)

prompt made failure modes localized and traceable — fixing FA-001 required changing only the triage prompt, not the entire system.

Pydantic output parsing eliminated the majority of format failures. Coupling the LLM output to a strict schema and implementing a single retry chain was sufficient for all 5 scenarios.

The evaluation harness was built early and run after every prompt iteration. This made prompt changes verifiable rather than speculative, and prevented regressions — a lesson directly applicable to production LLM systems.

Prompt engineering vs. model capability. The triage prompt arc (v1→v4) was measured on two models. On a small model (`gpt-5.4-nano`) prompt engineering alone lifted triage accuracy from 60% to 100% — v3 fixed over-escalation (FA-001) and v4 added trend awareness (FA-004). On the larger `gpt-4o` the model already reasons all five scenarios correctly from v1, so the same prompts act as an explicit, auditable safety guardrail rather than an accuracy lift. The clearest lesson of the project: careful prompt engineering matters most for smaller/cheaper models and remains a correctness guardrail for stronger ones.

8.2 What Was Difficult

Prompt version discipline: The most challenging aspect was not writing good prompts, but documenting *why* each change was made. The failure analysis log was essential — without it, the same bugs would have been re-introduced during v3 iteration.

Confidence calibration: Getting the synthesis agent to assign meaningfully different confidence scores across scenarios required explicit threshold anchors in the prompt. Without the 0.8/0.5/<0.5 scale, the agent defaulted to high confidence regardless of data completeness.

Module	**Deliverable**	**Implementation Evidence**
M1: Intro to LLMs	Model selection rationale	`portfolio/model_selection.md`: GPT-4o vs. GPT-4o-mini baseline experiments; per-agent temperature rationale
M2: LLM Application Design	Agent interface specifications	Pydantic v2 schemas: `RPMAlert`, `TriageDecision`, `EHRContext`, `AnamnesisSummary`, `ClinicalContextBrief`
M3: Prompt Content	Versioned prompt library	Prompt files across v1→v4 (triage to v4, others to v3); few-shot examples; chain-of-thought designs; `PydanticOutputParser` schema enforcement
M4: Conversational Agency	Multi-step reasoning chains	Triage: 5-step CoT; Synthesis: 6-step CoT; Anamnesis: 5-step CoT with `SENSITIVE_FLAG` handling
M5: Testing LLM Apps	Evaluation + observability	`evaluation/harness.py` (5 scenarios, end-to-end + retrieval/completeness metrics) and `evaluation/judge.py` (LLM-as-judge for the qualitative Ch. 9.1 metrics); **LangFuse** traces with token usage and cost
M6: Advanced LangChain	RAG pipeline	`rag/ingest.py` + `rag/retriever.py`: ChromaDB, `text-embedding-3-small`, chunk=512, overlap=64, k=5
M7: Autonomous Agents	Tools + memory	LangChain `@tools` (`vector_store_search`, `alert_classification`, `rpm_trend`, `parse_anamnesis`) in `agents/tools.py`; per-patient entity memory in `agents/memory.py`
M8: Multi-Agent Systems	Orchestrator with safety guardrails	`orchestrator.py`: **LangGraph** `StateGraph` (parallel fan-out/fan-in), CRITICAL escalation path, inter-agent schema contracts, session audit log

Table 11: Course module coverage

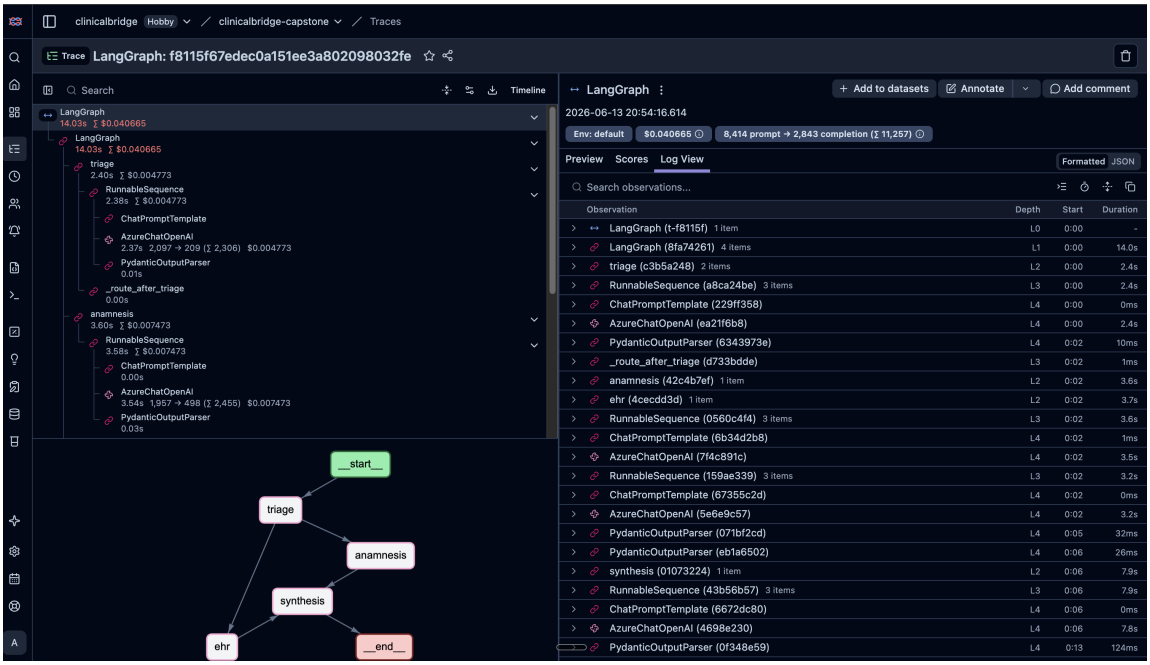

Figure 2: LangFuse observability: one trace per run (total cost $0.0407, 11,257 tokens). The tree shows the LangGraph nodes triage → `_route_after_triage` → EHR || Anamnesis → synthesis with per-agent latency, token usage, and USD cost; the inset graph view shows the orchestration topology.

Conflicting data: The `conflicting_data` scenario was the hardest to handle correctly. The model's tendency to "pick a winner" between contradictory sources was only corrected in v3 with an explicit rule requiring both claims to be stated and flagged.

8.3 Known Limitations

- **Hallucination risk is non-zero:** Guardrails reduce but do not eliminate hallucination. Clinical deployment would require additional validation layers.
- **Simulated data:** Real EHR records contain abbreviations, inconsistencies, and missing fields that simulated data underrepresents.
- **No real-time integration:** The system processes alerts in batch; integration with live RPM streams would require an event-driven architecture.
- **Agent memory scope:** A per-patient entity memory is implemented (`agents/memory.py`), but its scope is per-session in the current demonstration; richer cross-session conversation/-summary memory remains future work.
- **Synthesis clinical accuracy (∼89%):** The one metric just under target (90%). Two prompt variants and a denoised, spec-aligned judge were tried (FA-005); the score is a stable ceiling near the high 80s for solid-but-imperfect briefs and is reported honestly rather than forced. Real clinical use would need a stronger validation layer.

9 Ethical Considerations

- **Not a clinical tool.** This prototype must never be used for actual clinical decision-making. All outputs are contextual summaries to support, not replace, clinician judgment.

Figure 3: Automated evaluation harness output (`evaluation/harness.py v4`): 5/5 scenarios pass; triage 100%, hallucination 0%, source traceability 100%, anamnesis completeness 100%, retrieval precision 93% / recall 100%, average time-to-brief ≈14 s.

- **No real patient data.** Every record in the dataset is entirely simulated and fictional.
- **No diagnoses.** All agents are explicitly prohibited from stating diagnoses. The word "diagnosis" is banned from system prompts and monitored in outputs.
- **Human-in-the-loop.** The system is designed as decision support. The clinician always makes the final decision. CRITICAL alerts trigger immediate human escalation with no synthesis delay.
- **Transparency.** Every CCB includes confidence scores, source citations, and explicit uncertainty flags. The system is designed to surface what it does not know.

References

[1] Wright, A., et al. (2015). Problem list completeness in electronic health records: A multi-site study and assessment of success factors. *International Journal of Medical Informatics*.

[2] Hravnak, M., et al. (2018). Cardiorespiratory instability before and after implementing an integrated monitoring system. *Critical Care Medicine*.

[3] Senbekov, M., et al. (2020). The recent progress and applications of digital technologies in healthcare: A review. *International Journal of Environmental Research and Public Health*.

[4] Adler-Milstein, J., & Jha, A. K. (2017). HITECH Act drove large gains in hospital electronic health record adoption. *Health Affairs*.

[5] Vegesna, A., et al. (2017). Remote patient monitoring via non-invasive digital technologies: A systematic review. *Telemedicine and e-Health*.

[6] Cvach, M. (2012). Monitor alarm fatigue: An integrative review. *Biomedical Instrumentation & Technology*.

```
CLINICAL CONTEXT BRIEF  –  conflicting_data (PT-011)

Urgency: URGENT    Overall confidence: 0.8

Contextual Analysis:
  The RPM alert indicates systolic BP of 164.8 and diastolic BP of 101.2, both
  significantly above baseline thresholds. EHR data confirms a history of
  essential hypertension and triple antihypertensive therapy (Valsartan,
  Chlorthalidone, Amlodipine). However, prior notes and lab findings suggest
  possible non-adherence to medications, despite the patient's self-reported

Recommended Actions:
  1. Schedule an urgent follow-up appointment to evaluate BP control and
     adherence.
     source: RPM: systolic BP 164.8, diastolic BP 101.2; EHR: visit note 2025-1
     confidence: 0.9
  2. Perform a detailed medication adherence review, including pharmacy refill
     records and pill count.
     source: EHR: Glipizide drug level; Anamnesis: medication_adherence
     confidence: 0.8

Uncertainties & Gaps:
  [CONFLICTING_INFO] Discrepancy between self-reported adherence and pharmacy refill
     records.
  [MISSING_DATA] Objective measures of adherence beyond Glipizide drug level not
     found.
  [MISSING_DATA] Details on pharmacy refill records not fully specified.
```

Figure 4: Actual end-to-end system output — Clinical Context Brief for the `conflicting_data` scenario (PT-011). Note the `CONFLICTING_INFO` flag raised where the patient's self-reported adherence contradicts the sub-therapeutic drug level, with cited sources and confidence per action.

[7] Noah, B., et al. (2018). Impact of remote patient monitoring on clinical outcomes: An updated meta-analysis of randomized controlled trials. *NPJ Digital Medicine*.

[8] Bates, D. W., et al. (2014). Big data in health care: Using analytics to identify and manage high-risk and high-cost patients. *Health Affairs*.

[9] Bachmann, M., & Windak, A. (2020). Digital anamnesis in primary care: Potential and limitations. *MDPI Healthcare*.

[10] Singhal, K., et al. (2023). Large language models encode clinical knowledge. *Nature*.

[11] Nori, H., et al. (2023). Capabilities of GPT-4 on medical competence examinations. *arXiv preprint*.

[12] Thirunavukarasu, A. J., et al. (2023). Large language models in medicine. *Nature Medicine*.

[13] Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.

[14] Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR*.

[15] Park, J. S., et al. (2023). Generative agents: Interactive simulacra of human behavior. *UIST*.

[16] LangChain Documentation. Retrieval-Augmented Generation and Agent Frameworks. `https://docs.langchain.com`